

# GeeksforGeeks

A computer science portal for geeks

## GeeksQuiz

- [Home](#)
- [Algorithms](#)
- [DS](#)
- [GATE](#)
- [Interview Corner](#)
- [Q&A](#)
- [C](#)
- [C++](#)
- [Java](#)
- [Books](#)
- [Contribute](#)
- [Contests](#)
- [Jobs](#)

[Array](#)

[Bit Magic](#)

[C/C++](#)

[Articles](#)

[GFactS](#)

[Linked List](#)

[MCQ](#)

[Misc](#)

[Output](#)

[String](#)

[Tree](#)

[Graph](#)

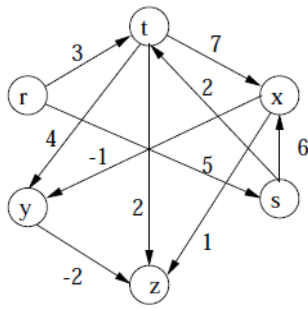
## Shortest Path in Directed Acyclic Graph

Given a Weighted Directed Acyclic Graph and a source vertex in the graph, find the shortest paths from given source to all other vertices.

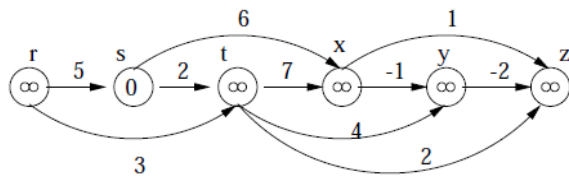
For a general weighted graph, we can calculate single source shortest distances in  $O(VE)$  time using [Bellman–Ford Algorithm](#). For a graph with no negative weights, we can do better and calculate single source shortest distances in  $O(E + V\log V)$  time using [Dijkstra's algorithm](#). Can we do even better for Directed Acyclic Graph (DAG)? We can calculate single source shortest distances in  $O(V+E)$  time for DAGs. The idea is to use [Topological Sorting](#).

We initialize distances to all vertices as infinite and distance to source as 0, then we find a topological sorting of the graph. [Topological Sorting](#) of a graph represents a linear ordering of the graph (See below, figure (b) is a linear representation of figure (a) ). Once we have topological order (or linear representation), we one by one process all vertices in topological order. For every vertex being processed, we update distances of its adjacent using distance of current vertex.

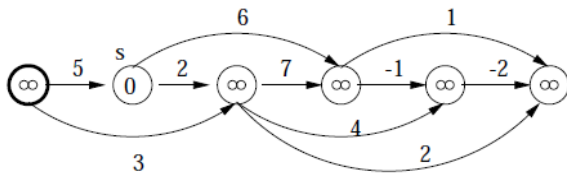
Following figure is taken from [this](#) source. It shows step by step process of finding shortest paths.



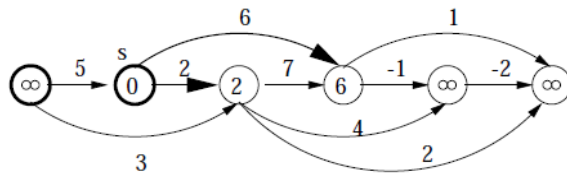
(a)



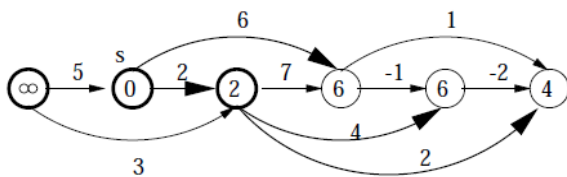
(b)



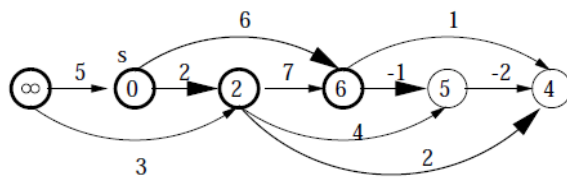
(c)



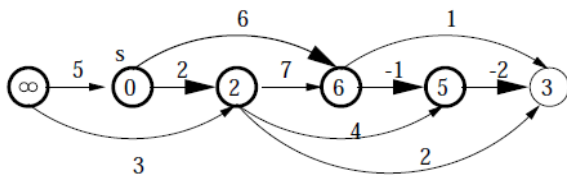
(d)



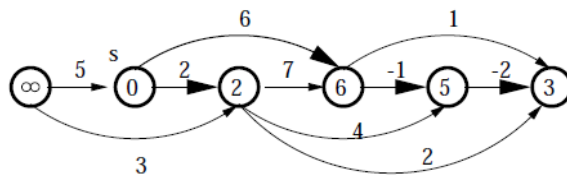
(e)



(f)



(g)



(h)

Following is complete algorithm for finding shortest distances.

1) Initialize  $\text{dist}[] = \{\text{INF}, \text{INF}, \dots\}$  and  $\text{dist}[s] = 0$  where  $s$  is the source vertex.

2) Create a topological order of all vertices.

3) Do following for every vertex  $u$  in topological order.

.....Do following for every adjacent vertex  $v$  of  $u$

.....if ( $\text{dist}[v] > \text{dist}[u] + \text{weight}(u, v)$ )

..... $\text{dist}[v] = \text{dist}[u] + \text{weight}(u, v)$

// A C++ program to find single source shortest paths for Directed Acyclic

```
#include<iostream>
```

```
#include <list>
```

```
#include <stack>
```

```
#include <limits.h>
```

```
#define INF INT_MAX
```

```
using namespace std;
```

// Graph is represented using adjacency list. Every node of adjacency list

// contains vertex number of the vertex to which edge connects. It also

// contains weight of the edge

```
class AdjListNode
```

```
{
```

```
    int v;
```

```

    int weight;
public:
    AdjListNode(int _v, int _w) { v = _v; weight = _w;}
    int getV() { return v; }
    int getWeight() { return weight; }
};

// Class to represent a graph using adjacency list representation
class Graph
{
    int V;    // No. of vertices'

    // Pointer to an array containing adjacency lists
    list<AdjListNode> *adj;

    // A function used by shortestPath
    void topologicalSortUtil(int v, bool visited[], stack<int> &Stack);
public:
    Graph(int V);    // Constructor

    // function to add an edge to graph
    void addEdge(int u, int v, int weight);

    // Finds shortest paths from given source vertex
    void shortestPath(int s);
};

Graph::Graph(int V)
{
    this->V = V;
    adj = new list<AdjListNode>[V];
}

void Graph::addEdge(int u, int v, int weight)
{
    AdjListNode node(v, weight);
    adj[u].push_back(node); // Add v to u's list
}

// A recursive function used by shortestPath. See below link for details
// http://www.geeksforgeeks.org/topological-sorting/
void Graph::topologicalSortUtil(int v, bool visited[], stack<int> &Stack)
{
    // Mark the current node as visited
    visited[v] = true;

    // Recur for all the vertices adjacent to this vertex
    list<AdjListNode>::iterator i;
    for (i = adj[v].begin(); i != adj[v].end(); ++i)
    {
        AdjListNode node = *i;
        if (!visited[node.getV()])
            topologicalSortUtil(node.getV(), visited, Stack);
    }

    // Push current vertex to stack which stores topological sort
    Stack.push(v);
}

```

}

// The function to find shortest paths from given vertex. It uses recursive  
 // topologicalSortUtil() to get topological sorting of given graph.

```
void Graph::shortestPath(int s)
```

{

```
    stack<int> Stack;
```

```
    int dist[V];
```

```
    // Mark all the vertices as not visited
```

```
    bool *visited = new bool[V];
```

```
    for (int i = 0; i < V; i++)
```

```
        visited[i] = false;
```

```
    // Call the recursive helper function to store Topological Sort
```

```
    // starting from all vertices one by one
```

```
    for (int i = 0; i < V; i++)
```

```
        if (visited[i] == false)
```

```
            topologicalSortUtil(i, visited, Stack);
```

```
    // Initialize distances to all vertices as infinite and distance
```

```
    // to source as 0
```

```
    for (int i = 0; i < V; i++)
```

```
        dist[i] = INF;
```

```
    dist[s] = 0;
```

```
    // Process vertices in topological order
```

```
    while (Stack.empty() == false)
```

{

```
        // Get the next vertex from topological order
```

```
        int u = Stack.top();
```

```
        Stack.pop();
```

```
        // Update distances of all adjacent vertices
```

```
        list<AdjListNode>::iterator i;
```

```
        if (dist[u] != INF)
```

{

```
            for (i = adj[u].begin(); i != adj[u].end(); ++i)
```

```
                if (dist[i->getV()] > dist[u] + i->getWeight())
```

```
                    dist[i->getV()] = dist[u] + i->getWeight();
```

}

}

```
    // Print the calculated shortest distances
```

```
    for (int i = 0; i < V; i++)
```

```
        (dist[i] == INF)? cout << "INF " : cout << dist[i] << " ";
```

}

```
// Driver program to test above functions
```

```
int main()
```

{

```
    // Create a graph given in the above diagram. Here vertex numbers are
```

```
    // 0, 1, 2, 3, 4, 5 with following mappings:
```

```
    // 0=r, 1=s, 2=t, 3=x, 4=y, 5=z
```

```
    Graph g(6);
```

```
    g.addEdge(0, 1, 5);
```

```
    g.addEdge(0, 2, 3);
```

```

g.addEdge(1, 3, 6);
g.addEdge(1, 2, 2);
g.addEdge(2, 4, 4);
g.addEdge(2, 5, 2);
g.addEdge(2, 3, 7);
g.addEdge(3, 4, -1);
g.addEdge(4, 5, -2);

int s = 1;
cout << "Following are shortest distances from source " << s << " \n";
g.shortestPath(s);

return 0;
}

```

Output:

```

Following are shortest distances from source 1
INF 0 2 6 5 3

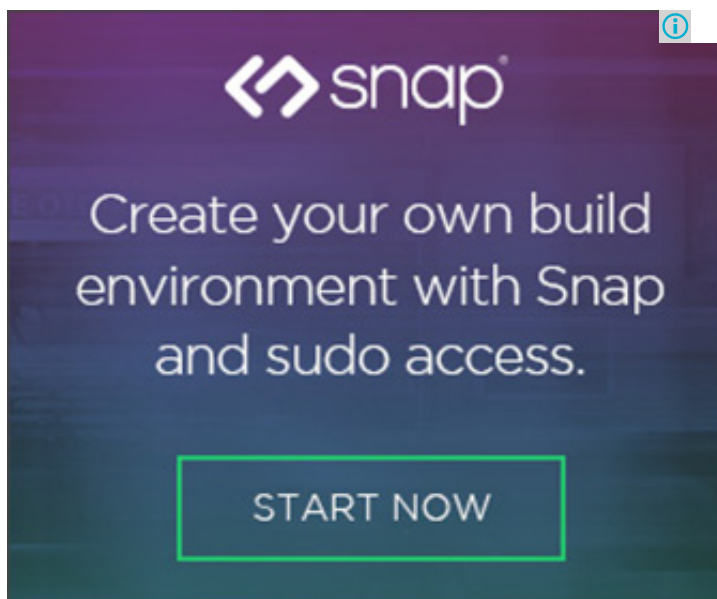
```

**Time Complexity:** Time complexity of topological sorting is  $O(V+E)$ . After finding topological order, the algorithm process all vertices and for every vertex, it runs a loop for all adjacent vertices. Total adjacent vertices in a graph is  $O(E)$ . So the inner loop runs  $O(V+E)$  times. Therefore, overall time complexity of this algorithm is  $O(V+E)$ .

References:

<http://www.utdallas.edu/~sizheng/CS4349.d/l-notes.d/L17.pdf>

Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above



## Related Topics:

- [Karger's algorithm for Minimum Cut | Set 1 \(Introduction and Implementation\)](#)
- [Greedy Algorithms | Set 9 \(Boruvka's algorithm\)](#)
- [Assign directions to edges so that the directed graph remains acyclic](#)
- [K Centers Problem | Set 1 \(Greedy Approximate Algorithm\)](#)
- [Find the minimum cost to reach destination using a train](#)

- [Applications of Breadth First Traversal](#)
- [Optimal read list for given number of days](#)
- [Print all paths from a given source to a destination](#)

Tags: [Graph](#)

Like { 47

Tweet

{ 4

+1

{ 3

**Writing code in comment?** Please use [ideone.com](http://ideone.com) and share the link here.

6 Comments

GeeksforGeeks

 Login ▾ Recommend 2 Share

Sort by Newest ▾



Join the discussion...

**WP** • 3 months ago

The link for topological sorting is broken.

1 ^ | ▾ • Reply • Share ›

**Aditya Goel** • 4 months ago<http://ideone.com/eaitQz>

^ | ▾ • Reply • Share ›

**Hector** • 6 months ago

Hello,

I am not sure what version of c you are using, but normally a list does not have a [] operator (line adj[u])

you probably want to be using a vector or a hash (not really standard) for the adj type.

^ | ▾ • Reply • Share ›

**Anurag Singh** ➔ Hector • 6 months ago

adj is not a list here.

adj is an array of list.

It is declared as:

list&lt;adjlistnode&gt; \*adj;

adj = new list&lt;adjlistnode&gt;[V];

so adj[0] will have 1st list, adj[1] will have 2nd list and so on.

^ | ▾ • Reply • Share ›

**jinzhi chen** • a year ago

wow, thanks

1 ^ | ▾ • Reply • Share ›

**Allanian** • 2 years ago

Explanation as great as longest in path for DAG. Thank you.

1 ^ | ▾ • Reply • Share ›

 Subscribe Add Disqus to your site Privacy

**GeeksforGeeks**

Like

99,723 people like GeeksforGeeks.



Feedback social plugin



dreamfactory

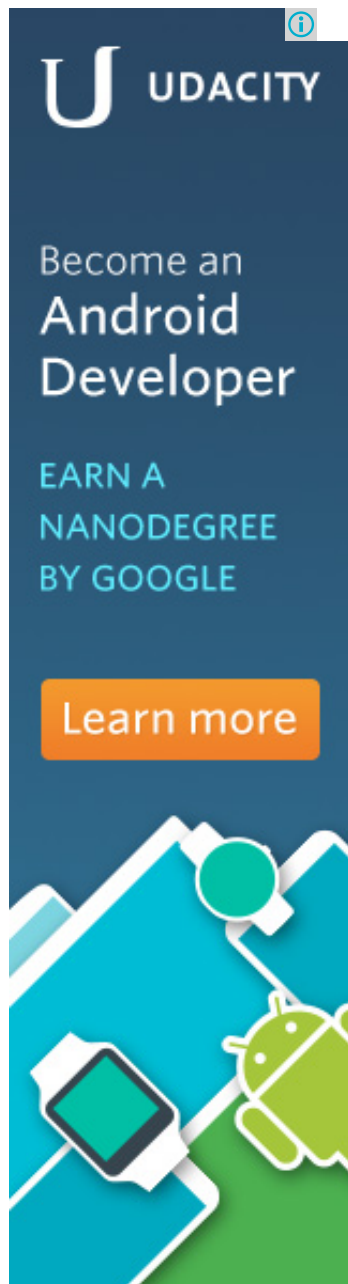
**Free Open Source  
REST API Platform**

Install on any server.  
Connect to SQL or NoSQL DB.  
Develop HTML5 or Native mobile.

**get started**

- [Interview Experiences](#)
- [Advanced Data Structures](#)
- [Dynamic Programming](#)
- [Greedy Algorithms](#)
- [Backtracking](#)
- [Pattern Searching](#)
- [Divide & Conquer](#)
- [Mathematical Algorithms](#)
- [Recursion](#)
- [Geometric Algorithms](#)





## • Popular Posts

- [All permutations of a given string](#)
- [Memory Layout of C Programs](#)
- [Understanding “extern” keyword in C](#)
- [Median of two sorted arrays](#)
- [Tree traversal without recursion and without stack!](#)
- [Structure Member Alignment, Padding and Data Packing](#)
- [Intersection point of two Linked Lists](#)
- [Lowest Common Ancestor in a BST.](#)
- [Check if a binary tree is BST or not](#)
- [Sorted Linked List to Balanced BST](#)

• [Follow @GeeksforGeeks](#)

## • Recent Comments

- [Belly](#)

we put extern explicitly for C variables when...

[What are the default values of static variables in C?](#) · 12 minutes ago

- [Naveen Kumar](#)

<http://ideone.com/FLD3LN>

[Bottom View of a Binary Tree](#) · 17 minutes ago

- [Phani Theja Swarup Vempalli](#)

An easy to understand approach similar to...

[Maximum of all subarrays of size k \(Added a O\(n\) method\)](#) · 22 minutes ago

- [Ahmed marzouk](#)

thanks alot

[Memory Layout of C Programs](#) · 1 hour ago

- [Belly](#)

Very good article

[Understanding “extern” keyword in C](#) · 1 hour ago

- [pk](#)

void printLevelOrder(struct node\* root) { node\*...

[Level Order Tree Traversal](#) · 1 hour ago

**Ads by Google**

► [Graphs](#)

► [Geeks for Geeks](#)

• ► [Algorithm](#)

**Ads by Google**

► [Graph in Java](#)

► [C++ Problem](#)

► [Vertex Distance](#)

**Ads by Google**

► [Geeks](#)

► [Int](#)

► [Path](#)

@geeksforgeeks, [Some rights reserved](#) \_\_\_\_ [Contact Us!](#) \_\_\_\_ [Abut Us!](#)

Powered by [WordPress](#) & [MooTools](#), customized by geeksforgeeks